

Notas Práticas e Exercícios 4

Disciplina: Programação Orientada a Objetos

{*Anotações para uso pessoal}

Sumário

1	Composição e Agregação	1
2	Herança com Composição e Agregação	2
3	Abstract e Final	5
4	Exercícios	6

1 Composição e Agregação

Para exemplificação adotaremos a classe A (por exemplo, conta) como a parte e a classe B como o todo (por exemplo, agência). Na composição, a classe B (**classe todo**) cria e gerencia um objeto da classe A (**classe parte**) como um atributo privado. O objeto da classe A é criado quando a instância da classe B é criada e tem seu ciclo de vida vinculado à instância da classe B. Ou seja, **não temos um método setter para o objA em B**. Isso significa que quando a instância da classe B é destruída, o objeto da classe A também é destruído.

A composição representa uma relação forte entre as classes, indicando que a classe A é uma parte integral da classe B e não pode existir independentemente dela.

Composição

```
1 public class A {
2     // Classe A com seus atributos e métodos
3 }
4
5 public class B {
6     private A objA = new A(); // Composição com a classe A
7
8     // Outros atributos e métodos da classe B
9 }
```

Listing 1: Composição entre A e B

Na agregação, a classe B tem um atributo privado do tipo A, mas o objeto da classe A não é criado dentro da classe B. Em vez disso, o objeto da classe A é fornecido externamente, **geralmente por meio de um método "setter"**.

A agregação representa uma relação mais fraca entre as classes, indicando que a classe A pode existir independentemente da classe B. A classe B apenas mantém uma referência ao objeto da classe A e não é responsável por seu ciclo de vida.

Agregação

```
1 public class A {
2     // Classe A com seus atributos e métodos
3 }
4
5 public class B {
6     private A objA; // Agregação com a classe A
7
8     public void setObjA(A objA) {
9         this.objA = objA;
10    }
11
12    // Outros atributos e métodos da classe B
13 }
```

Listing 2: Agregação entre A e B

2 Herança com Composição e Agregação

Composição

Ao herdar da classe B, a classe C tem acesso ao objeto referenciado pelo atributo objA quando o atributo é declarado como protected. A classe C pode acessar e utilizar esse objeto da classe A, e é possível chamar os métodos públicos e protegidos do objeto A a partir da classe C.

No exemplo a seguir, a classe C acessa o objeto da classe A referenciado pelo atributo objA:

```
1 public class A {
2     // Classe A com seus atributos e métodos
3
4     public void doSomething() {
5         System.out.println("Doing something in class A");
6     }
7 }
8
9 public class B {
10    protected A objA new A();
11
12    // Outros atributos e métodos da classe B
13 }
14
15 public class C extends B {
16    // Classe C herda de B e tem acesso ao atributo 'b' do tipo A
17
18    public void useA() {
19        objA.doSomething();
20    }
21 }
```

```

20     }
21 }
22
23 public class Main {
24     public static void main(String[] args) {
25         C objC = new C();
26         objC.useA(); // "Doing something in class A" será printado
27     }
28 }

```

Neste exemplo, a classe C tem acesso ao objeto da classe A referenciado pelo atributo objA, e pode chamar o método doSomething() do objeto A. O programa principal (main) cria uma instância da classe C e chama o método useA(), que por sua vez chama o método doSomething() do objeto A.

Caso o atributo b fosse declarado como private em B, C herdaria os comportamentos e atributos da classe B. No entanto, como o atributo objA é privado, ele não pode ser acessado diretamente pela classe C.

Agregação

Se a relação entre as classes B e A for uma agregação, o objeto da classe A não será criado dentro da classe B, mas será fornecido externamente e associado à classe B por meio de um método "setter". A classe C ainda herda a estrutura da classe B e tem acesso ao atributo objA do tipo A, desde que seja declarado como protected.

```

1 public class A {
2     // Classe A com seus atributos e métodos
3
4     public void doSomething() {
5         System.out.println("Doing something in class A");
6     }
7 }
8
9 public class B {
10     protected A objA; // Agregação com a classe A
11
12     public void setObjA(A objA) {
13         this.objA = objA;
14     }
15
16     // Outros atributos e métodos da classe B
17 }
18
19 public class C extends B {
20     // Classe C herda de B e tem acesso ao atributo 'b' do tipo A
21
22     public void useA() {
23         if (b != null) {
24             b.doSomething();
25         } else {
26             System.out.println("A is not set in B");
27         }
28     }
29 }

```

```

29 }
30
31 public class Main {
32     public static void main(String[] args) {
33         A a = new A();
34         C c = new C();
35         c.setObjA(a);
36         c.useA(); // "Doing something in class A" will be printed
37     }
38 }

```

Neste exemplo, a relação entre as classes B e A é de agregação, então a classe B não cria um objeto da classe A internamente. Em vez disso, o objeto A é criado no programa principal (main) e associado à classe B (e, por extensão, à classe C que herda de B) usando o método "setter" `setObjA()`. A classe C ainda pode acessar e utilizar o objeto da classe A referenciado pelo atributo `objA`.

Essa abordagem também funcionaria para a composição entre as classes B e A, como no exemplo anterior. A classe C herda a estrutura da classe B e tem acesso ao objeto da classe A referenciado pelo atributo `objA`, independentemente de a relação entre as classes B e A ser de agregação ou composição.

No exemplo a seguir, a classe B tem um atributo privado `objA` do tipo A e utiliza agregação. A classe C herda de B e, para acessar o objeto referenciado por `objA`, usamos um método "getter" na classe B. O mesmo ocorreria em uma composição quando for usado `private` para os atributos da classe pai.

```

1 public class A {
2     // Classe A com seus atributos e métodos
3
4     public void doSomething() {
5         System.out.println("Doing something in class A");
6     }
7 }
8
9 public class B {
10     private A objA; // Agregação com a classe A (atributo privado)
11
12     public void setObjA(A objA) {
13         this.objA = objA;
14     }
15
16     // Método "getter" para acessar o atributo privado 'b'
17     public A getB() {
18         return objA;
19     }
20
21     // Outros atributos e métodos da classe B
22 }
23
24 public class C extends B {
25     // Classe C herda de B
26
27     public void useA() {
28         A a = getB();
29         if (a != null) {
30             a.doSomething();

```

```

31         } else {
32             System.out.println("A is not set in B");
33         }
34     }
35 }
36
37 public class Main {
38     public static void main(String[] args) {
39         A a = new A();
40         C c = new C();
41         c.setObjA(a);
42         c.useA(); // "Doing something in class A" will be printed
43     }
44 }

```

Neste exemplo, a classe B utiliza agregação com a classe A e mantém um atributo privado b do tipo A. A classe C herda de B, mas não pode acessar o atributo privado b diretamente. Para contornar isso, a classe B fornece um método "getter" getB() que retorna o objeto da classe A referenciado por b. A classe C utiliza o método "getter" para acessar o objeto da classe A e chamar o método doSomething() nele.

3 Abstract e Final

Em Java, os modificadores *abstract* e *final* são usados para impor restrições em classes e métodos.

- **Abstract:** A palavra-chave *abstract* é usada para indicar que uma classe ou método não pode ser instanciado ou implementado diretamente. Esses elementos servem como base para outras classes ou métodos e devem ser estendidos ou sobrescritos para serem utilizados.

- *Classe Abstract:* Uma classe abstrata não pode ser instanciada diretamente. Ela é projetada para ser herdada por outras classes. Uma classe abstrata pode conter métodos abstratos e não abstratos. As classes filhas que herdam uma classe abstrata devem fornecer implementações para todos os métodos abstratos da classe base, a menos que a classe filha também seja declarada como abstrata.

```

1 // Classe abstrata Animal
2 public abstract class Animal {...}

```

- *Método Abstract:* Um método abstrato é declarado sem implementação na classe base. A implementação desse método deve ser fornecida pelas classes filhas que herdam a classe base. Métodos abstratos só podem existir em classes abstratas.

```

1 // Método abstrato: as subclasses de Animal devem fornecer uma
  implementação para este método
2 public abstract void fazerSom();

```

- **Final:** A palavra-chave *final* é usada para indicar que um elemento não pode ser modificado. Quando aplicada a uma classe ou método, impede a herança ou a sobrescrita, respectivamente.
- *Classe Final:* Uma classe final não pode ser estendida, ou seja, não é possível criar classes filhas a partir dela. Isso é útil quando se deseja criar uma classe imutável ou quando a classe já possui todas as funcionalidades necessárias e não deve ser modificada por herança.

```

1 // Classe final Veiculo
2 public final class Veiculo { ...}

```

- *Método Final*: Um método final não pode ser sobrescrito pelas classes filhas. Isso é útil quando se deseja garantir que a implementação de um método específico não seja alterada por subclasses. Dessa forma, é possível manter um comportamento consistente em toda a hierarquia de classes.

```

1 // Método final: este método não pode ser sobrescrito pelas
  subclasses de Animal
2 public final void dormir() {
3     System.out.println("O animal está dormindo.");
4 }

```

4 Exercícios

1. Classe Animal: Crie uma classe chamada `Animal` com os métodos abstratos `fazerSom()` e `seMovimentar()`. Crie subclasses como `Mamifero`, `Ave`, `Reptil`, etc., e implemente os métodos de acordo com o comportamento específico de cada classe. Use polimorfismo para chamar esses métodos em um array de objetos `Animal`.
2. Calculadora de formas geométricas: Crie uma classe chamada `FormaGeometrica` com métodos abstratos para calcular a área e o perímetro. Crie subclasses como `Circulo`, `Retangulo`, `Triangulo`, etc., e implemente os métodos específicos para cada forma. Utilize polimorfismo para calcular e exibir a área e o perímetro de diferentes formas geométricas.
3. Sistema de pagamento: Crie uma classe chamada `Funcionario` com atributos como nome e salário. Crie métodos abstratos para calcular o salário de diferentes tipos de funcionários, como `Horista`, `Assalariado` e `Comissionado`. Implemente as subclasses e use polimorfismo para calcular o salário de cada funcionário em um array de objetos `Funcionario`.
4.
 - **Passo 1:** Crie uma classe chamada `ContaBancaria` com os seguintes atributos e métodos:
 - `numeroDaConta`: int
 - `saldo`: double
 - `depositar(double valor)`: Adiciona o valor ao saldo.
 - `sacar(double valor)`: Subtrai o valor do saldo, se possível.
 - **Passo 2:** Crie uma classe chamada `ContaPoupanca` que estende `ContaBancaria`:
 - Adicione um atributo `taxaDeJuros`: double
 - Crie um construtor que recebe `numeroDaConta`, `saldo` e `taxaDeJuros` como parâmetros e use a palavra-chave `super` para inicializar os atributos da classe `ContaBancaria`.
 - Adicione um método `aplicarRendimento()`: Multiplica o saldo pelo `taxaDeJuros` e adiciona o resultado ao saldo atual.
 - **Passo 3:** Crie uma classe chamada `ContaCorrente` que estende `ContaBancaria`:
 - Adicione um atributo `limite`: double
 - Crie um construtor que recebe `numeroDaConta`, `saldo` e `limite` como parâmetros e use a palavra-chave `super` para inicializar os atributos da classe `ContaBancaria`.
 - Sobrescreva o método `sacar(double valor)`: Subtrai o valor do saldo, se possível, considerando o limite disponível. Isto é, caso o saldo não seja suficiente, use o valor do limite disponível (se for suficiente) e deixe o saldo negativo.

- **Passo 4:** Crie uma classe Principal com o método main:
 - Instancie um objeto ContaPoupanca e um objeto ContaCorrente.
 - Utilize casting implícito para armazenar os objetos ContaPoupanca e ContaCorrente em um array de ContaBancaria.
 - Percorra o array e chame o método sacar() e depositar() de cada conta, utilizando casting quando necessário para acessar os métodos específicos de cada tipo de conta.

Exercício: Implementar, Identificar e Corrigir Erros em um Sistema de Gestão de Funcionários

Neste exercício, você receberá um trecho de código que implementa um sistema de gestão de funcionários usando herança, polimorfismo e casting. Sua tarefa é implementar o código, identificar e corrigir os erros no código para que ele funcione corretamente.

```
1 // Arquivo Funcionario.java
2 public abstract class Funcionario {
3     private String nome;
4     private double salario;
5
6     public Funcionario(String nome, double salario) {
7         this.nome = nome;
8         this.salario = salario;
9     }
10
11     public final String getNome(){
12         return nome;
13     }
14     public abstract double calcularBonus();
15 }
16
17 // Arquivo Gerente.java
18 public class Gerente extends Funcionario {
19     private double bonus;
20     private String area;
21
22     public Gerente(String nome, double salario, double bonus, String area) {
23         super(nome, salario);
24         this.bonus = bonus;
25     }
26
27     public double calcularBonus() {
28         return salario + bonus;
29     }
30
31     public String getNome(){
32         System.out.println("Nome: " + super.getNome());
33         return null;
34     }
35
36     public String getArea(){
37         return this.area;
38     }
39 }
```

```

40
41 // Arquivo Vendedor.java
42 public class Vendedor extends Funcionario {
43     private double comissao;
44
45     public Vendedor(String nome, double salario, double comissao) {
46         super.Funcionario(nome, salario);
47         this.comissao = comissao;
48
49     }
50
51     public double calcularBonus() {
52         return salario + comissao;
53     }
54
55 }
56
57 // Arquivo Principal.java
58 public class Principal {
59     public static void main(String[] args) {
60         Funcionario[] funcionarios = new Funcionario[3];
61         funcionarios[0] = new Gerente("Maria", 5000, 1000);
62         funcionarios[1] = new Vendedor("João", 3000, 500);
63         funcionarios[2] = new Vendedor("Lucas", 2500, 400);
64         funcionarios[3] = new Funcionario("Lucas", 2500, 400);
65
66         for (Funcionario funcionario : funcionarios) {
67             System.out.println("Nome: " + funcionario.getNome());
68             System.out.println("Area: " + funcionario.getArea());
69             System.out.println("Bonus: " + funcionario.calcularBonus());
70         }
71     }
72 }
73 }

```